BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG

BÁO CÁO

TIN HỌC LÝ THUYẾT

ĐỀ TÀI

DEMO XÂY DỰNG NFA-e VÀ KIỂM TRA CHUỖI ĐẦU VÀO

Giáo viên hướng dẫn:

Ths. Phạm Xuân Hiền

Sinh viên thực hiện:

Họ và tên:	Nguyễn Hoàng Anh Tuấn
MSSV:	B2106865
Họ và tên:	Nguyễn Thanh Hiếu
MSSV:	B2106834

Cần Thơ, 2023

NHẬN XÉT CỦA GIẢNG VIÊN

MỤC LỤC

PHẦN A: PHẦN GIỚI THIỆU 1
I. Đặt vấn đề: 1
II. Mục tiêu bài toán: 1
PHẦN B: PHẦN NỘI DUNG 2
I. Mô tả bài toán: 2
1. Phương pháp thực hiện. 2
II. Thiết kế và cài đặt: 3
1. Thiết kế: 3
2. Cài đặt thuật toán: 4
PHẦN C: PHẦN KẾT LUẬN 9
I. Kết quả đạt được: 9
II. Hướng phát triển: 10
TÀI LIỆU THAM KHẢO 11
PHỤ LỤC 12
Hướng dẫn cài đặt đường dẫn cho thư viện “graphviz”. 12

PHẦN A: PHẦN GIỚI THIỆU

I. Đặt vấn đề:

Bài toán xây dựng NFAε là một vấn đề trong lĩnh vực tự động hóa và ngôn ngữ hình thức. Đây là một bài toán quan trọng trong việc mô hình hóa và xử lý các ngôn ngữ hình thức. Liên quan đến thiết kế các trình biên dịch, xử lý ngôn ngữ tự nhiên và nhiều ứng dụng khác.

II. Mục tiêu bài toán:

a. Mục tiêu:

- Xác định xem 1 chuỗi đầu vào có thuộc ngôn ngữ được biểu diễn bởi NFAε hay không: Theo dõi tất cả các đường đi có thể thông qua NFAε để xem liệu có ít nhất một đường đi nào chấp nhận chuỗi đó hay không.
- Xác định ngôn ngữ được biểu diễn bởi NFAε: Xác định tập hợp các chuỗi mà NFAε có thể chấp nhận. Điều này đòi hỏi việc xây dựng 1 NFA hoặc DFA tương đương để biểu diễn ngôn ngữ được chấp nhận bởi NFAε ban đầu.

b. Ứng dụng:

- Mô hình hóa các quy tắc của 1 hệ thống:
  - Hệ thống đặt vé máy bay.
  - Hệ thống ngân hàng.
  - Hệ thống bán lẻ.
- Xử lý ngôn ngữ tự nhiên:
  - Phân tích cú pháp dưới dạng trạng thái và chuyển trạng thái.
  - Tạo mẫu ngôn ngữ bằng cách biểu diễn các mẫu ngôn ngữ phổ biến dưới dạng các trạng thái và chuyển trạng thái.
  - Tìm kiếm các cụm từ hoặc biểu thức trong văn bản bằng cách biểu diễn các cụm từ hoặc biểu thức dưới dạng trạng thái, chuyển trạng thái.
- Tự động hóa kiểm tra:
  - Tạo các trường hợp kiểm tra tự động cho các phần mềm.
  - Tự động hóa việc thực thi các trường hợp kiểm tra.
  - Tự động hóa phân tích kết quả kiểm tra các phần mềm.
- Thiết kế phần mềm:

- Kiểm tra hệ thống có đáp ứng các yêu cầu của người dùng.
- Mô hình hóa các hành vi của phần mềm.

PHẦN B: PHẦN NỘI DUNG

I. Mô tả bài toán:

1. Phương pháp thực hiện.

Có 3 phương pháp phổ biến để thực hiện xây dựng NFAε và kiểm tra 1 chuỗi có thuộc NFAε không.

a. Phương pháp đệ quy:

Nếu trạng thái hiện tại là trạng thái chấp nhận thì chuỗi đầu vào được chấp nhận. Nếu không thì xem xét tất cả các chuyển trạng thái có nhãn ε từ trạng thái hiện tại. Đối với mỗi chuyển trạng thái ε, gọi thuật toán đệ quy với trạng thái mới là trạng thái đích của chuyển trạng thái.

Nếu đệ quy trả về True thì chuỗi được chấp nhận.

b. Phương pháp dựa trên bảng:

Khởi tạo 1 bảng với kích thước bằng số trạng thái ε.Đối với mỗi trạng thái của NFAε, đánh dấu trạng thái đó không được chấp nhận.

Đối với mỗi trạng thái chấp nhận, đánh dấu trạng thái đó là được chấp nhận.

Đối với mỗi trạng thái của NFAε, xem xét tất cả các chuyển trạng thái có nhãn ký tự từ trạng thái đó.

Đối với mỗi chuyển trạng thái có nhãn ký tự, cập nhật bảng để chỉ ra rằng trạng thái đích của chuyển trạng thái là được chấp nhận nếu và chỉ nếu trạng thái hiện tại là được chấp nhận và ký tự nhãn của chuyển trạng thái là một phần của chuỗi đầu vào.

Kiểm tra xem trạng thái hiện tại có được chấp nhận hay không.

c. Phương pháp lặp

Khởi tạo trạng thái hiện tại là trạng thái khởi đầu.

Lặp qua danh sách đầu vào, chuyển trạng thái dựa trên giá trị đầu vào. Kiểm tra xem trạng thái hiện tại có thuộc tập trạng thái chấp nhận không. Nếu trạng thái hiện tại có thuộc tập trạng thái chấp nhận thì chuỗi đầu vào được chấp nhận.

❖ So sánh 3 phương pháp thực hiện:

Phương pháp	Ưu điểm	Hạn chế
Đệ quy	- Đơn giản và dễ hiểu. - Có thể thực hiện bất kỳ NFAε nào.	- Tốn nhiều thời gian cho các NFAε lớn.
Bảng	- Hiệu quả hơn phương pháp đệ quy cho các NFAε lớn.	- Khó hiểu và khó sử dụng hơn phương pháp đệ quy
Lặp	- Đơn giản và dễ hiểu. - Hiệu quả cho các NFAε nhỏ.	- Tốn nhiều thời gian cho các NFAε lớn.

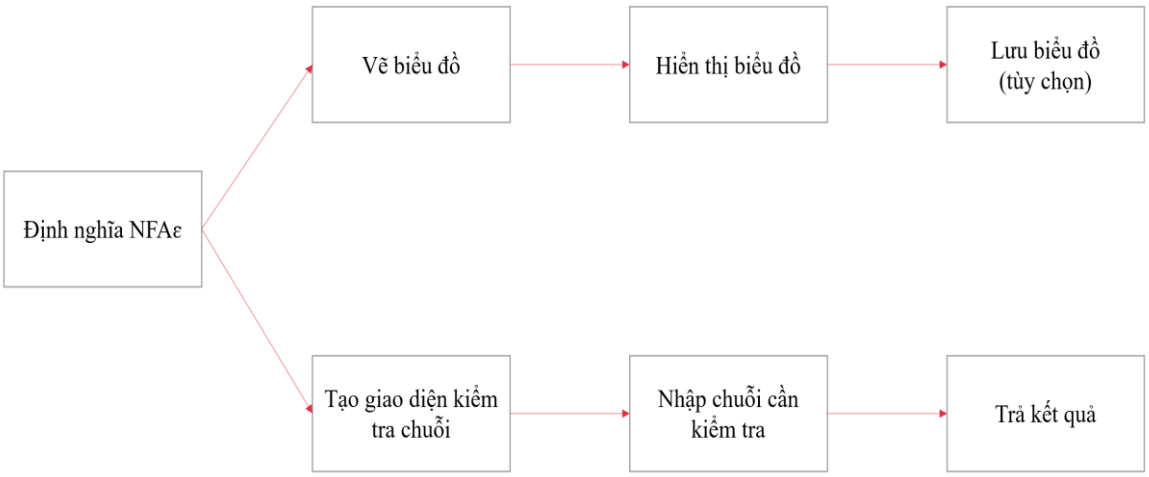
II. Thiết kế và cài đặt:

Sử dụng **phương pháp lặp** để thiết kế và cài đặt chương trình xây dựng NFAε và kiểm tra chuỗi có thuộc NFAε đó không.

1. Thiết kế:

1. Định nghĩa lớp "NFAε.
2. Hàm "demo".
3. Hàm "check_input_string".
4. Xây dựng NFAε.
5. Chạy hàm "demo" và "check_input_string".

Các bước thực hiện:



2. Cài đặt thuật toán:

Sử dụng 2 thư viện: "**graphviz**"- sử dụng để tạo biểu đồ, "**tkinter**" -sử dụng để tạo giao diện nhập chuỗi đầu vào cần kiểm tra.

a. Cài đặt, định nghĩa lớp NFAε.

- Tạo 1 lớp "**NFAε**" và khởi tạo các thuộc tính:

```
class NFAe(object):
    def __init__(self, states, alphabet, transition_function, start_state, accept_states):
        self.states = states
        self.alphabet = alphabet
        self.transition_function = transition_function
        self.start_state = start_state
        self.accept_states = accept_states
        self.current_states = {start_state}
```

- **"states":** Tập trạng thái.
- **"alphabet":** Bộ chữ cái.
- **"transition_function":** Hàm chuyển trạng thái.
- **"start_state":** Trạng thái bắt đầu.
- **"accept_states":** Trạng thái chấp nhận (kết thúc).
- **"current_state":** Trạng thái hiện tại.

Sử dụng **"self.current_state**" để theo dõi trạng thái hiện tại. Trạng thái hiện tại là tập hợp các trạng thái mà NFAε đang ở trong quá trình xử lý chuỗi đầu vào. Có vai trò quan trọng để lưu trữ trạng thái hiện tại trong quá trình chuyển đổi trạng thái dựa trên đầu vào.

- Các phương thức thuộc lớp NFAε:

```
def transition_to_states_with_input(self, input_value):
        new_current_states = set()
        states_to_process = set(self.current_states)

        while states_to_process:
            state = states_to_process.pop()

            transitions = self.transition_function.get((state, input_value), set())
            new_current_states.update(transitions)

            epsilon_transitions = self.transition_function.get((state, ''), set())
            states_to_process.update(epsilon_transitions)

        self.current_states = new_current_states
```

Hàm transition_to_states_with_input(): Dùng để chuyển từ trạng thái này sang trạng thái tiếp theo bằng giá trị input.

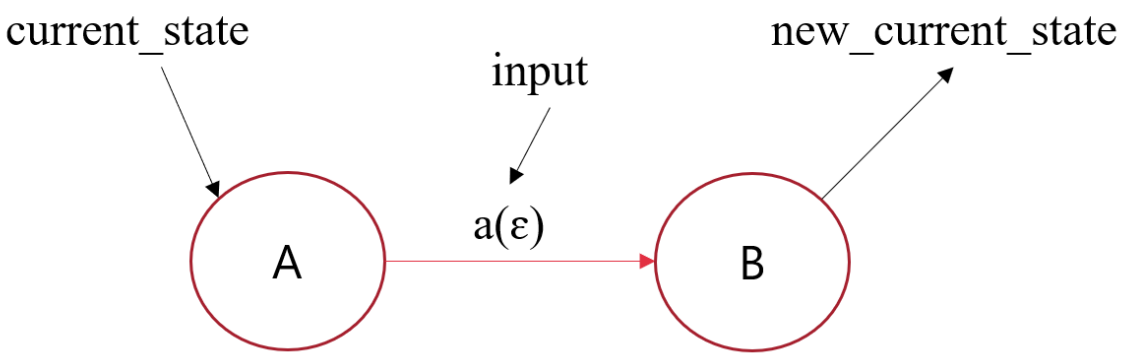
- new_current_state: Tạo 1 tập hợp chứa các trạng thái mới sau khi chuyển trạng thái với đầu vào.
- states_to_process: là 1 tập hợp chứa các trạng thái hiện tại của NFAε. Trong quá trình lặp, chúng ta sẽ xử lý các trạng thái trong tập này.

Lặp cho đến khi tập states_to_process rỗng. Trong mỗi vòng lặp chúng ta sẽ xử lý 1 trạng thái từ tập này. Lấy 1 trạng thái từ tập hợp states_to_process và gán giá trị bằng state. Lấy ngẫu nhiên 1 trạng thái và đảm bảo mỗi trạng thái chỉ được xử lý 1 lần và không bị lặp lại. Lấy tất cả các trạng thái mà NFAε có thể chuyển đến từ state với đầu vào input_value. Kết quả được lưu trong tập transitions. Sau mỗi lần chuyển, cập nhật lại new_current_states bằng cách thêm các trạng thái từ transitions.

Lấy tất cả các trạng thái mà NFAε có thể chuyển đến từ **state** với đầu vào rỗng (ε). Kết quả lưu trong epsilon_transitions.

Thêm trạng thái từ epsilon_transitions vào states_to_process để xử lý chúng trong các vòng lặp tiếp theo.

Cuối cùng, cập nhật trạng thái hiện tại của NFAε current_states với các trạng thái mới tích lũy trong new_current_states.




```
def in_accept_state(self):
        return bool(self.current_states.intersection(self.accept_states))
```

Hàm in_accept_state(): dùng để kiểm tra xem trạng thái hiện tại có trạng thái chấp nhận hay không. Dùng phương pháp intersection của set() để kiểm tra và trả về 1 tập hợp chứa các phần tử chung trong trạng thái hiện tại current_states và trạng thái chấp nhận accept_states.

Kết quả hàm trả về là 1 giá trị bool. Nếu có trạng thái chung thì hàm trả về kết quả True. Ngược lại, nếu không có trạng thái chung thì sẽ trả về False.

```
def go_to_initial_state(self):
        self.current_states = {self.start_state}
```

Hàm go_to_initial_state(): Gán giá trị của tập hiện tại thành tập trạng thái khởi đầu. Đảm bảo có thể kiểm tra nhiều chuỗi khác nhau mà không cần 1 đối tượng NFAε mới và khởi tạo lại từ đầu.

```
def run_with_input_list(self, input_list):
        self.go_to_initial_state()
        for inp in input_list:
            self.transition_to_states_with_input(inp)

        return self.in_accept_state()
```

Hàm run_with_input_list(): kiểm tra 1 chuỗi đầu vào có thuộc NFAε không.

- Đầu tiên, gọi hàm **go_to_initial_state** để đặt lại trạng thái ban đầu.
- Duyệt qua danh sách đầu vào **input_list**. Với mỗi giá trị **inp**, gọi hàm **transition_to_states_with_input** để chuyển trạng thái. Tức là đọc từng ký tự đầu vào và di chuyển từ trạng thái này sang trạng thái khác.
- Cuối cùng, hàm kiểm tra trạng thái hiện tại có đang ở trạng thái chấp nhận không. Nếu đang ở trạng thái chấp nhận thì hàm trả về **True**. Ngược lại, hàm trả về **False.**

b. Hàm vẽ biểu đồ NFAε.

```
def demo(nfae):
    dot = graphviz.Digraph(format='png')

    for state in nfae.states:
        if state in nfae.accept_states:
            dot.node(state, shape='doublecircle')
        else:
            dot.node(state)

    for (from_state, input_symbol), to_states in nfae.transition_function.items():
        for to_state in to_states:
            label = input_symbol if input_symbol != '' else 'ε'
            dot.edge(from_state, to_state, label)

    dot.render('nfae_diagram')

    dot.view('nfae_diagram')
```

Tạo 1 đối tượng biểu đồ bằng graphviz.Digraph(format='png') được sử dụng để vẽ biểu đồ.

Duyệt qua tất cả các trạng thái state trong nfae.states và thêm vào biểu đồ. Nếu trạng thái đó thuộc trạng thái chấp nhận (nfae.accept_states) thì vẽ trạng thái đó bằng hình tròn kép. Nếu không thì vẽ hình tròn đơn.

Duyệt qua tất cả các chuyển trạng thái (nfae.transition_function) và thêm cạnh vào biểu đồ. Các cạnh này thể hiện đường đi từ trạng thái ban đầu (from_state) đến trạng thái đích (to_state). Nhãn của cạnh là giá trị đầu vào (input_symbol) hoặc ε.

Sau khi xây dựng hoàn chỉnh thì hàm lưu biểu đồ dưới dạng hình ảnh (PNG) bằng dot.render('nfae_diagram'). Lúc này sẽ tạo ra 1 tệp hình ảnh có tên nfae_diagram trong thư mục làm việc.

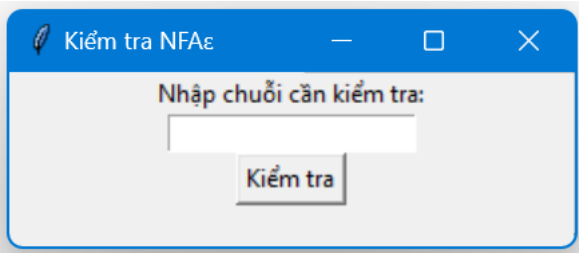
Cuối cùng sử dụng dot.view('nfae_diagram') để hiển thị ảnh.

c. Hàm để tạo 1 giao diện cho người dùng nhập vào 1 chuỗi cần kiểm tra có thuộc NFAε hay không.

```
def check_input_string():
    def check_string():
        input_string = input_entry.get()
        result = nfae.run_with_input_list(list(input_string))
        if result:
            result_label.config(text=f"Chuỗi '{input_string}' được chấp nhận bởi NFAε.")
        else:
            result_label.config(text=f"Chuỗi '{input_string}' không được chấp nhận bởi NFAε.")

    window = tk.Tk()
    window.title("Kiểm tra NFAε")

    input_label = tk.Label(window, text="Nhập chuỗi cần kiểm tra:")
    input_label.pack()

    input_entry = tk.Entry(window)
    input_entry.pack()

    check_button = tk.Button(window, text="Kiểm tra", command=check_string)
    check_button.pack()

    result_label = tk.Label(window, text="")
    result_label.pack()

    window.mainloop()
```

Hàm check_string(): lấy chuỗi từ ô nhập liệu và kiểm tra chuỗi bằng cách gọi hàm run_with_input_list.

Cửa sổ giao diện nhập liệu:



Dựa vào kết quả kiểm tra, hàm cập nhật nhãn **result_lable** để hiển thị kết quả cho người dùng.

d. Hàm main

```
f=open("nfae.txt",'r')
states= f.readline().split()
alphabet=f.readline().split()
input = f.readline().split()
start_state= [input[0]]
accept_states=[input[1]]
t=f.readlines()
tf=dict()
for i in t:
    i=i.replace( __old: '\n', __new: '')
    a=i.split(' ')
    tf[(a[0],a[1])]=a[2:len(a)]
nfae = NFAe(states, alphabet, tf, start_state, accept_states)

demo(nfae)
check_input_string(nfae)
```

Đọc từ file, tạo 1 NFAε có các trạng thái, bộ chữ cái, trạng thái bắt đầu và chấp nhận.

Sau đó thêm bảng chuyển trạng thái tf là 1 tự điển với khóa cặp là (‘trạng thái bắt đầu’, ‘ký tự đầu vào’) và trạng thái có thể chuyển đến.

Tạo 1 đối tượng nfae với các thuộc tính trên.

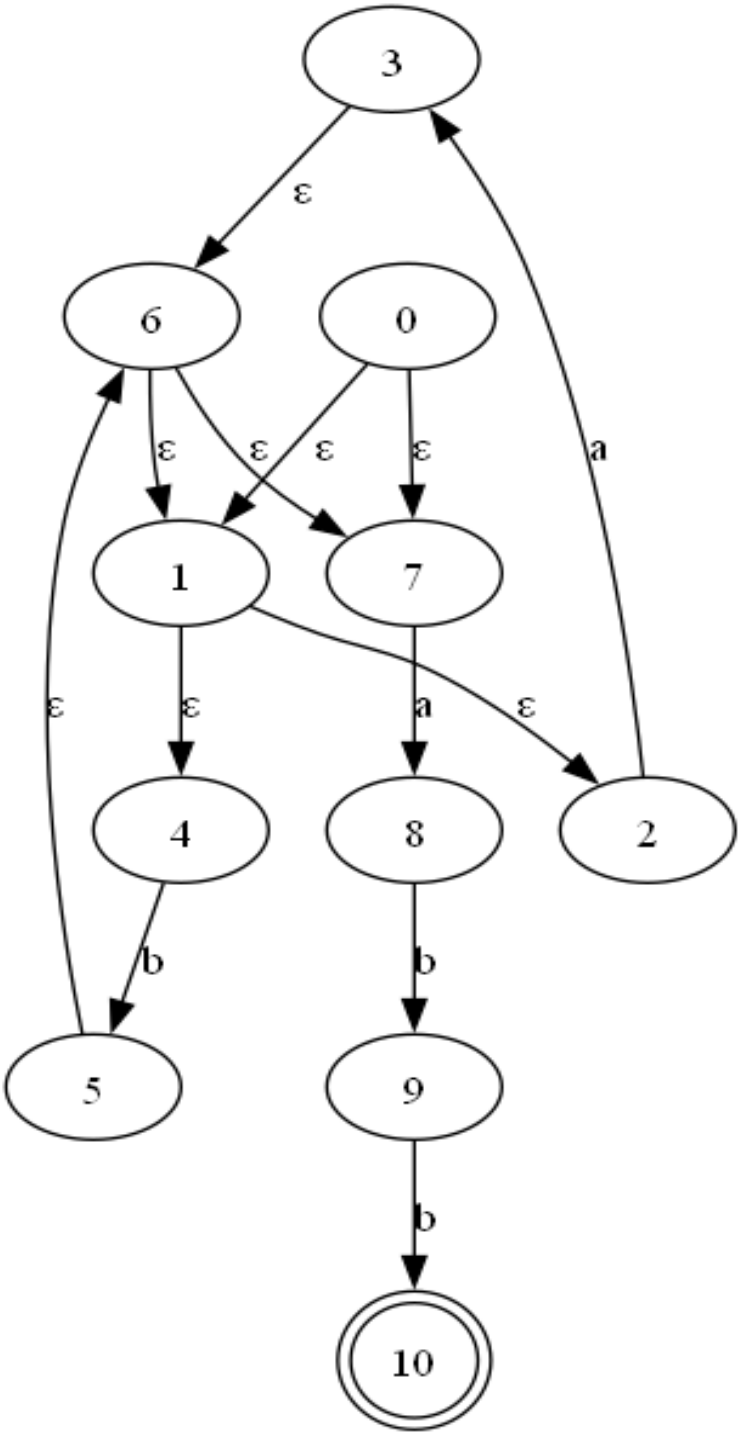
Gọi hàm demo(nfae) và check_input_string() để thực thi chương trình.

PHẦN C: PHẦN KẾT LUẬN

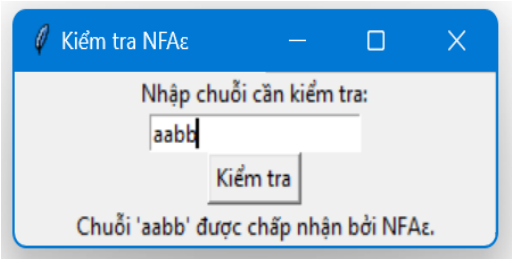
I. Kết quả đạt được:

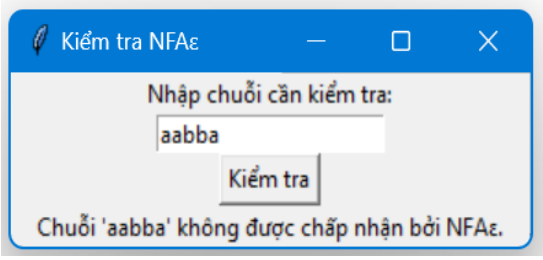
Sau khi chạy chương trình trên sẽ cho ra kết quả như sau:

- Xuất ra 1 hình ảnh thể hiện biểu đồ NFAε đã định nghĩa.



- Tạo 1 cửa sổ để người dùng nhập chuỗi cần kiểm tra. Nhập chuỗi để kiểm tra và trả về kết quả.





II. <u>Hướng phát triển:</u>

- Tối ưu hóa NFAε: Giảm độ phức tạp của việc xử lý. Tối ưu hóa có thể loại bỏ trạng thái không cần thiết, chuyển đổi ε không cần thiết… để biểu diễn cùng 1 ngôn ngữ với ít trạng thái hơn.
- Tìm hiểu tính mạnh mẽ và giới hạn của NFAε: Nghiên cứu tính mạnh mẽ và giới hạn của NFAε trong việc biểu diễn các ngôn ngữ và quá trình xử lý. Giúp hiểu rõ các ưu điểm, hạn chế của NFAε.
- Xây dựng thuật toán chuyển đổi NFAε thành DFA: Thuật toán chuyển đổi này giúp dễ dàng xử lý và kiểm tra chuỗi đầu vào.
- Mở rộng NFAε cho các ngôn ngữ đặc biệt: Để mô hình hóa các ngôn ngữ đặc biệt như ngôn ngữ chính quy với bao đóng Kleene, ngôn ngữ không có epsilon và nhiều khía cạnh khác của lý thuyết tự động hóa.
- Sử dụng NFAε cho các ứng dụng cụ thể: Như trình soạn thảo ngôn ngữ tự nhiên, kiểm tra tính hợp lệ cho biểu thức chính quy hoặc xác định ngôn ngữ của 1 trang web.

TÀI LIỆU THAM KHẢO

1. Võ Huỳnh Trâm, Giáo trình "*Tin học lý thuyết*", Trường CNTT&TT, Đại học Cần Thơ, 2009.

2. Nguyễn Văn Định, *"Automat và ngôn ngữ hình thức"*, Bộ Giáo dục và Đào tạo, Trường ĐH Nông Nghiệp Hà Nội.

3. Nguyễn Văn Định, *"Thiết kế và phân tích thuật toán",* ĐH Khoa học Tự nhiên, ĐHQG Hà Nội, 2000.

4. Đặng Huy Ruận, *"Lý thuyết ngôn ngữ hình thức và Otomat"*, NXB Đại hoc Quốc gia Hà Nội, 2002.

5. Phan Đình Diệu, *"Lý thuyết Otomat và Thuật toán"*, NXB Đại học và trung học chuyên nghiệp, Hà Nội 1971.

6. Nguyễn Văn Ba, Giáo trình "*Ngôn ngữ và hình thức*", Trường Đại học Bách khoa Hà Nội, 1994.

7. Nguyễn Thanh Tùng, "*Lý thuyết ngôn ngữ hình thức và Ôtômát*", Trường Đại học Bách khoa Tp.Hồ Chí Minh, 1993.

PHỤ LỤC

Hướng dẫn cài đặt đường dẫn cho thư viện "graphviz".

1) Tải thư viện **"graphviz"**

https://gitlab.com/api/v4/projects/4207231/packages/generic/graphviz-releases/9.0.0/windows_10_cmake_Release_graphviz-install-9.0.0-win64.exe

2) Chạy chương trình đã tải về.

3) Chú ý click vào ô tạo đường dẫn cho biến môi trường **"Add Graphviz to the system PATH for all users."**

4) Sau khi cài đặt xong restart chương trình Python và chạy.

(Có thể tải thư viện "graphviz" bằng lệnh *pip* hoặc *conda* và thêm đường dẫn vào biến môi trường)